

Python小练习：从多文件夹提取指定列并整合

作者：凯鲁嘎吉 - 博客园 <http://www.cnblogs.com/kailugaji/>

1. 问题陈述

这是一个懒人创造世界的一个例子。在科研实验、机器学习项目或自动化测试中，我们经常会遇到这样的场景：一个主文件夹下有多个子文件夹，每个子文件夹中都有类似结构的CSV文件，我们需要从中提取特定列的数据，然后将所有数据整合到一个Excel文件中进行对比分析。

想象一下这样一个典型的工作流：

- 有5个不同的实验设置，每个设置在独立的文件夹中运行
- 每个实验生成一个progress_eval.csv文件记录进度与实验结果
- 你需要从每个文件中提取"accuracy"这一列
- 最后将所有数据放在一个Excel表格中对比，并进行后续处理(如取平均值、标准差、标准误)

传统的手动操作需要：逐个打开5个CSV文件 → 找到目标列 → 复制数据 → 粘贴到Excel → 调整格式 → 用Excel函数(如AVERAGE()、STDEV.S())处理数据... 这个过程不仅耗时，而且容易出错。本文使用Python来一键完成这个繁琐的过程！

2. Python代码

这个脚本主要包含以下几个核心功能：

- 批量提取数据：从指定数量的子文件夹中提取CSV文件的特定列
- 智能列匹配：支持按列名或列索引提取数据
- 数据对齐：自动将不同长度的数据对齐到相同行数
- 交互式操作：提供友好的命令行交互界面

```
# Python代码：从多文件夹提取指定列并整合
# 这段代码的主要功能是：
# 从多个实验文件夹中批量提取指定CSV文件的特定列数据，
# 并自动对齐整合为统一的Excel表格，便于后续分析比较
# 作者：凯鲁嘎吉 - 博客园 http://www.cnblogs.com/kailugaji/
```

```
import pandas as pd
from pathlib import Path
from typing import Optional, Union
```

```
def extract_data(
    base_path: str,
    num_folders: int,
    column_spec: Union[str, int],
    csv_name: str = "progress_eval.csv"
) -> None:
```

```
    """
    从文件夹中提取指定CSV文件的列数据
```

```
    参数：
```

```
    base_path: 主文件夹路径
    num_folders: 处理的子文件夹数量
    column_spec: 列名(str)或列索引(int)
    csv_name: 已有的需要处理的CSV文件名
```

```
    """
```

```
    path = Path(base_path)
```

```

if not path.exists():
    print(f"文件夹 {base_path} 不存在")
    return

# 获取子文件夹
folders = sorted([f for f in path.iterdir() if f.is_dir()][:num_folders])
if not folders:
    print("未找到子文件夹")
    return

data = {}
print(f"处理 {csv_name} 文件: ")

for folder in folders:
    csv_file = folder / csv_name
    print(f" {folder.name}", end="")

    if not csv_file.exists():
        print(" -> 文件不存在")
        continue

    try:
        df = pd.read_csv(csv_file)
        col = _get_column(df, column_spec)
        if col:
            data[folder.name] = df[col]
            print(f" -> 提取 {len(df)} 行")
    except Exception as e:
        print(f" -> 错误: {e}")

if not data:
    print("没有提取到数据")
    return

# 对齐并保存数据
aligned_df = _align_to_max(data)
output_file = f"{path.name}_seed{num_folders}_{_col_str(column_spec)}.xlsx" # 可以自行改输出的表格文件名
aligned_df.to_excel(output_file, index=False)
print(f"\n数据已保存到 {output_file}")
print(f"包含 {len(aligned_df.columns)} 列, {len(aligned_df)} 行")

def _get_column(df: pd.DataFrame, spec: Union[str, int]) -> Optional[str]:
    """获取目标列名"""
    if isinstance(spec, str):
        # 精确匹配
        if spec in df.columns:
            return spec
        # 模糊匹配
        matches = [col for col in df.columns if spec.lower() in col.lower()]
        return matches[0] if matches else None

    if isinstance(spec, int):
        return df.columns[spec] if 0 <= spec < len(df.columns) else None

    return None

def _align_to_max(data: dict) -> pd.DataFrame:
    """对齐数据到最大行数"""
    max_len = max(len(d) for d in data.values())

    aligned_data = {}
    for name, series in data.items():
        if len(series) < max_len:
            # 明确指定dtype为object, 避免警告
            padding = pd.Series([None] * (max_len - len(series)), dtype=object)
            aligned_data[name] = pd.concat([series, padding], ignore_index=True)
        else:
            aligned_data[name] = series.reset_index(drop=True)

    return pd.DataFrame(aligned_data)

def _col_str(spec: Union[str, int]) -> str:
    """生成列标识字符串"""
    if isinstance(spec, str):

```

```

        return spec.replace('/', '_').replace('\\', '_')[:20]
    return f"col{spec}"

def show_columns(
    base_path: str,
    folder_idx: int = 0,
    csv_name: str = "progress_eval.csv"
) -> None:
    """显示列信息"""
    path = Path(base_path)
    folders = sorted([f for f in path.iterdir() if f.is_dir()])

    if folder_idx < len(folders):
        csv_file = folders[folder_idx] / csv_name
        if csv_file.exists():
            df = pd.read_csv(csv_file)
            print(f"\n{folders[folder_idx].name} 的列信息 ({csv_name}):")
            for i, col in enumerate(df.columns):
                print(f"    [{i}] {col}")
        else:
            print(f"文件 {csv_name} 不存在")

def select_column_interactive(
    base_path: str,
    folder_idx: int = 0,
    csv_name: str = "progress_eval.csv"
) -> Optional[Union[str, int]]:
    """交互式选择列"""
    show_columns(base_path, folder_idx, csv_name)

    while True:
        choice = input("\n输入列名/索引 (q退出): ").strip()
        if choice.lower() == 'q':
            return None

        if choice.isdigit():
            return int(choice)
        return choice

# 使用示例
if __name__ == "__main__":
    base_path = "Algorithm" # 主文件夹
    num_folders = 5 # 子文件夹的个数(前num_folders个子文件夹)
    csv_name = "progress_eval.csv" # 已有的需要处理的CSV文件名

    # 显示列信息
    show_columns(base_path, 0, csv_name)

    # 交互式选择
    selected = select_column_interactive(base_path, 0, csv_name)
    if selected is not None:
        extract_data(base_path, num_folders, selected, csv_name)

    # 或直接指定
    extract_data(base_path, num_folders, 3, csv_name)
    extract_data(base_path, num_folders, "acc", csv_name)
    extract_data(base_path, num_folders, "accuracy", csv_name)

```

3. 使用实例

假设我们有一个名为Algorithm的文件夹，结构如下：

```

Algorithm/
├── seed1/
│   └── progress_eval.csv
├── seed2/
│   └── progress_eval.csv
├── ...
├── seed5/
└── progress_eval.csv

```

每个CSV文件包含类似的数据列： epoch, train_loss, val_loss, accuracy, learning_rate, total_time

Algorithm文件夹：

Data (D:) > Python code > 2023.3 exercise > 0. Excel处理 > Algorithm >			
名称	修改日期	类型	大小
seed0	2026/1/13 10:49	文件夹	
seed1	2026/1/13 10:49	文件夹	
seed2	2026/1/13 10:49	文件夹	
seed3	2026/1/13 10:49	文件夹	
seed4	2026/1/13 10:49	文件夹	

每个子文件夹：

Data (D:) > Python code > 2023.3 exercise > 0. Excel处理 > Algorithm > seed0			
名称	修改日期	类型	大小
progress_eval.csv	2026/1/13 10:47	Microsoft Excel ...	1 KB

每个CSV文件内容(随便举的例子)：

	A	B	C	D	E	F	
1	epoch	train_loss	val_loss	accuracy	learning_rate	total_time	
2	0	100	80	0	0.5	10	
3	10	99	79	2	0.5	20	
4	20	98	78	4	0.5	30	
5	30	97	77	6	0.5	40	
6	40	96	76	8	0.5	50	
7	50	95	75	10	0.5	60	
8	60	94	74	12	0.5	70	
9	70	93	73	14	0.5	80	
10	80	92	72	16	0.5	90	
11	90	91	71	18	0.5	100	
12	100	90	70	20	0.5	110	

先显示所有列信息： show_columns(base_path, 0, csv_name)

结果：

```
seed0 的列信息 (progress_eval.csv):
[0] epoch
[1] train_loss
[2] val_loss
[3] accuracy
```

```
[4] learning_rate
[5] total_time
```

示例1：交互式使用

```
selected = select_column_interactive(base_path, 0, csv_name)
if selected is not None:
    extract_data(base_path, num_folders, selected, csv_name)
```

键入3，得到：

```
seed0 的列信息 (progress_eval.csv):
[0] epoch
[1] train_loss
[2] val_loss
[3] accuracy
[4] learning_rate
[5] total_time
```

```
输入列名/索引 (q退出): 3
处理 progress_eval.csv 文件:
seed0 -> 提取 11 行
seed1 -> 提取 11 行
seed2 -> 提取 11 行
seed3 -> 提取 11 行
seed4 -> 提取 11 行
```

数据已保存到 Algorithm_seed5_col3.xlsx
包含 5 列, 11 行

键入acc，得到：

```
seed0 的列信息 (progress_eval.csv):
[0] epoch
[1] train_loss
[2] val_loss
[3] accuracy
[4] learning_rate
[5] total_time
```

```
输入列名/索引 (q退出): acc
处理 progress_eval.csv 文件:
seed0 -> 提取 11 行
seed1 -> 提取 11 行
seed2 -> 提取 11 行
seed3 -> 提取 11 行
seed4 -> 提取 11 行
```

数据已保存到 Algorithm_seed5_acc.xlsx
包含 5 列, 11 行

示例2：直接指定

(1) `extract_data(base_path, num_folders, 3, csv_name)`

得到：

```
处理 progress_eval.csv 文件:
seed0 -> 提取 11 行
seed1 -> 提取 11 行
seed2 -> 提取 11 行
seed3 -> 提取 11 行
seed4 -> 提取 11 行
```

数据已保存到 Algorithm_seed5_col3.xlsx
包含 5 列, 11 行

(2) `extract_data(base_path, num_folders, "acc", csv_name)` 或
者 `extract_data(base_path, num_folders, "accuracy", csv_name)`

得到：

处理 progress_eval.csv 文件：

```
seed0 -> 提取 11 行
seed1 -> 提取 11 行
seed2 -> 提取 11 行
seed3 -> 提取 11 行
seed4 -> 提取 11 行
```

数据已保存到 Algorithm_seed5_acc.xlsx

包含 5 列, 11 行

得到的Excel表格内容就是将seed子文件夹下面的CSV中的精度那一列整合到一个表格中：

	A	B	C	D	E
1	seed0	seed1	seed2	seed3	seed4
2	0	0	0	0	0
3	2	2	2	2	2
4	4	4	4	4	4
5	6	6	6	6	6
6	8	8	8	8	8
7	10	10	10	10	10
8	12	12	12	12	12
9	14	14	14	14	14
10	16	16	16	16	16
11	18	18	18	18	18
12	20	20	20	20	20
13					

4. 将上述得到的Excel表格每一列取平均值、标准差、标准误（可选）

在上述基础上计算生成的Excel文件的前n列的平均值、标准差、标准误差

作者：凯鲁嘎吉 - 博客园 <http://www.cnblogs.com/kailugaji/>

```
import pandas as pd
import numpy as np
from pathlib import Path
```

```
def add_statistics_columns(excel_file: str, num_columns: int, output_suffix: str = "_with_stats") -> None:
```

```
    """
```

```
    处理Excel文件，计算前num_columns列的均值、标准差和标准误
```

```
    参数：
```

```
        excel_file: Excel文件路径
```

```
        num_columns: 计算前几列的统计量
```

```
        output_suffix: 输出文件名的后缀
```

```
    """
```

```
    # 检查文件是否存在
```

```
    file_path = Path(excel_file)
```

```
    if not file_path.exists():
```

```
        print(f"错误：文件 {excel_file} 不存在")
```

```
        return
```

```
    # 检查文件格式
```

```
    if file_path.suffix.lower() not in ['.xlsx', '.xls']:
```

```
        print(f"错误：{excel_file} 不是Excel文件")
```

```
        return
```

```

try:
    # 读取Excel文件
    df = pd.read_excel(excel_file)

    # 检查列数是否足够
    if len(df.columns) < num_columns:
        print(f"错误：表格只有 {len(df.columns)} 列，无法计算前 {num_columns} 列的统计量")
        return

    # 计算前num_columns列的统计量
    columns_to_analyze = df.columns[:num_columns]

    # 计算均值
    df['mean'] = df[columns_to_analyze].mean(axis=1, skipna=True)

    # 计算标准差
    df['std'] = df[columns_to_analyze].std(axis=1, skipna=True)

    # 计算标准误（标准差除以根号n）
    # 先计算每行的有效数据个数（排除NaN）
    valid_counts = df[columns_to_analyze].count(axis=1)
    df['sem'] = df['std'] / np.sqrt(valid_counts)

    # 可选：格式化显示，保留4位小数
    df['mean'] = df['mean'].round(4)
    df['std'] = df['std'].round(4)
    df['sem'] = df['sem'].round(4)

    # 生成输出文件名
    stem = file_path.stem
    suffix = file_path.suffix
    output_file = f"{stem}{output_suffix}{suffix}"

    # 保存到新文件
    df.to_excel(output_file, index=False)

    print(f"原始文件：{excel_file}")
    print(f"处理后的文件：{output_file}")
    print(f"已添加三列统计量：mean（均值）、std（标准差）、sem（标准误）")
    print(f"处理了 {len(df)} 行数据")

except Exception as e:
    print(f"处理文件时发生错误：{e}")

if __name__ == "__main__":
    excel_file = "Algorithm_seed5_acc.xlsx" # 需要处理的Excel文件名
    num_columns = 5 # 计算前num_columns列的统计量

    # 调用处理函数
    add_statistics_columns(excel_file, num_columns)

```

结果：

原始文件：Algorithm_seed5_acc.xlsx
 处理后的文件：Algorithm_seed5_acc_with_stats.xlsx
 已添加三列统计量：mean（均值）、std（标准差）、sem（标准误）
 处理了 11 行数据

	A	B	C	D	E	F	G	H
1	seed0	seed1	seed2	seed3	seed4	mean	std	sem
2	0	0	0	0	0	0	0	0
3	2	2	2	2	2	2	0	0
4	4	4	4	4	4	4	0	0
5	6	6	6	6	6	6	0	0
6	8	8	8	8	8	8	0	0
7	10	10	10	10	10	10	0	0
8	12	12	12	12	12	12	0	0
9	14	14	14	14	14	14	0	0
10	16	16	16	16	16	16	0	0
11	18	18	18	18	18	18	0	0
12	20	20	20	20	20	20	0	0

得到的新Excel文件在原先基础上多了三列：平均值、标准差、标准误。

5. 实际应用场景

(1) 机器学习实验分析

当你在不同随机种子下运行实验时，这个工具可以帮助你：比较不同随机种子的训练曲线、提取关键指标进行统计分析、生成整齐的数据表格用于论文图表

(2) 科学研究数据整理

处理重复实验数据时，可以：批量提取实验测量值、统一数据格式、准备数据用于进一步统计分析

这个脚本是一个实用的数据提取工具，特别适合需要处理多个相似数据文件的场景。通过自动化繁琐的文件操作，它能够：大幅提高数据处理效率、减少人为错误、提供一致的输出格式、支持灵活的列选择方式。